

SUMMARY

This topic covers a technical briefing on advanced Stack and Queue operations, including expression conversion, evaluation techniques, and the structural logic and applications of Queues. It highlights both theoretical principles and practical implementation strategies for competitive programming and system-level computation.

Stacks and Queues are linear data structures with specialized operational rules:

- **Expression Conversion:** Stack-based algorithms handle Infix to Postfix and Prefix conversions using operator priority and associativity. Right-to-left associative operators (e.g., exponentiation) require distinct handling.
- **Prefix Conversion Protocol:** Converting Infix to Prefix using a stack follows a three-step "Reverse-Process-Reverse" methodology.
- **Queue Evolution:** Simple array-based queues suffer from logical overflow, which Circular Queues resolve via modulo arithmetic for efficient memory usage.
- **Functional Applications:** Both structures are foundational in CPU scheduling, spooling, recursion, and expression evaluation in compilers.

1. EXPRESSION CONVERSION METHODOLOGIES

1.1 Infix to Postfix Conversion (Stack-Based)

- **Operands:** Added directly to output.
- **Operators:**
 - Push if the stack is empty.
 - Push if incoming operator has higher priority than the stack Top.
 - Pop stack to output if incoming operator has lower or equal priority until condition is met.
- **Associativity Twist:** Right-to-left operators (like $\wedge$) do not cause the stack to pop when encountering the same operator; they are pushed instead.

1.2 Infix to Prefix Conversion

1.2.1 Without Using a Stack

1. Identify the operator with highest priority according to associativity rules.
2. Rewrite as [Operator][Operand 1][Operand 2].
3. Repeat until entire expression is processed.

1.2.2 Using a Stack (Reversal Method)

1. Reverse the infix expression.
2. Swap parentheses: '(' becomes ')' and vice versa.
3. Apply standard postfix-like stack processing.

4. Reverse the output string to obtain correct Prefix notation.

2. EXPRESSION EVALUATION TECHNIQUES

2.1 Postfix Evaluation

1. Push operands onto the stack.
2. On encountering an operator, pop the top two operands: (Second Pop) [Operator] (First Pop).
3. Push result back onto the stack.
4. Remaining symbol after processing is the final answer.

2.2 Prefix Evaluation

1. Reverse the Prefix expression.
2. Process using the stack with the order: (First Pop) [Operator] (Second Pop).
3. The final result is obtained once the stack processing completes.

3. QUEUE DATA STRUCTURES AND IMPLEMENTATION

3.1 Basic Array Implementation

- **Front:** Index of element to be dequeued.
- **Rear:** Index of last inserted element.
- Initial State: Front = -1, Rear = -1. First insertion increments both to 0.

3.2 Limitation of Simple Queues

- Logical overflow occurs when $\text{Rear} = \text{Size} - 1$ even if there are free spaces at the start of the array.

3.3 Circular Queues

- **Enqueue:** $\text{Rear} = (\text{Rear} + 1) \% \text{Size}$.
- **Dequeue:** $\text{Front} = (\text{Front} + 1) \% \text{Size}$.
- **Full Condition:** $\text{Front} == (\text{Rear} + 1) \% \text{Size}$.

4. KEY APPLICATIONS AND ADVANCED VARIANTS

4.1 Applications

- CPU scheduling and process management.
- Printer spooling and shared resource management.
- Synchronization between fast and slow devices.
- Networking buffers and router queues.

4.2 Advanced Variants

- **Deque:** Insertion and deletion at both Front and Rear.
- **Priority Queue:** Elements processed according to assigned priority, typically implemented via Heaps.

5. INSTRUCTIONAL INSIGHTS AND KEY QUOTES

- "Consistency is the key... Every story has three points of view: yours, mine, and the truth."
- "Things aren't hard; you just need to follow the lecture to make them your favorite topic."
- "When things get reversed [as in Prefix conversion], 'friends' (operators of same priority) no longer remain friends."

